Pet Fish Simulator Code Analysis

Repo: https://eae-git.eng.utah.edu/u1537288/pcg-fish

Which PCG algorithm did you use? Why did you choose that?

I used a combination of Perlin noise and seeded grammar-based generation to create the fishbowl environment. Perlin noise was used to generate the sand floor height variation, which gives the environment a more natural and organic look compared to a flat plane. I chose this because it is lightweight, deterministic, and produces smooth transitions that resemble real terrain.

For object placement, I used a rule-based procedural system with constraints instead of a more complex algorithm like Wave Function Collapse. Each object type is spawned based on rules such as spacing, clustering, and priority order. This approach gave me more control over how objects appear in a small, contained environment like a fishbowl. Using a seed ensures that each generated level is reproducible, while still allowing for variety when the seed changes.

```cpp
void AProceduralSandFloor::BuildMesh()
{
    if (!ProceduralMesh || NumX < 1 || NumY < 1 || GridSpacing <= 0.0f)
    {
        return;
    }

    ProceduralMesh->ClearAllMeshSections();

    TArray<FVector> Vertices;
    TArray<int32> Triangles;
    TArray<FVector> Normals;
    TArray<FVector2D> UVs;
    TArray<FColor> VertexColors;
    TArray<FProcMeshTangent> Tangents;

    const int32 VertexCountX = NumX + 1;
    const int32 VertexCountY = NumY + 1;

    const float TotalWidth = NumX * GridSpacing;
    const float TotalHeight = NumY * GridSpacing;
    const float HalfWidth = TotalWidth * 0.5f;
    const float HalfHeight = TotalHeight * 0.5f;

    for (int32 Y = 0; Y < VertexCountY; Y++)
    {
        for (int32 X = 0; X < VertexCountX; X++)
        {
            const float XPos = (X * GridSpacing) - HalfWidth;
            const float YPos = (Y * GridSpacing) - HalfHeight;
            const float ZPos = GetHeightAtPoint(XPos, YPos);

            Vertices.Add(FVector(XPos, YPos, ZPos));
            UVs.Add(FVector2D(((float)X / (float)NumX) * UVScale, ((float)Y / (float)NumY) * UVScale));
            Normals.Add(FVector::UpVector);
            Tangents.Add(FProcMeshTangent(1.0f, 0.0f, 0.0f));
            VertexColors.Add(FColor::White);
        }
    }
```

```cpp
void AFishBowlGenerator::GenerateFromSeed(int32 InSeed)
{
    Seed = InSeed;
    RandomStream.Initialize(Seed);

    if (SandFloor)
    {
        SandFloor->Seed = Seed;
        SandFloor->GenerateMesh();
    }

    UE_LOG(LogTemp, Warning, TEXT("GenerateFromSeed called"));
    UE_LOG(LogTemp, Warning, TEXT("SpawnTypes: %d"), SpawnTypes.Num());

    ClearGeneratedContent();

    for (const FSpawnTypeData& SpawnType : SpawnTypes)
    {
        const int32 SpawnCount = GetSpawnCountForType(SpawnType);

        for (int32 i = 0; i < SpawnCount; ++i)
        {
            TryPlaceObject(SpawnType);
        }
    }
}
```

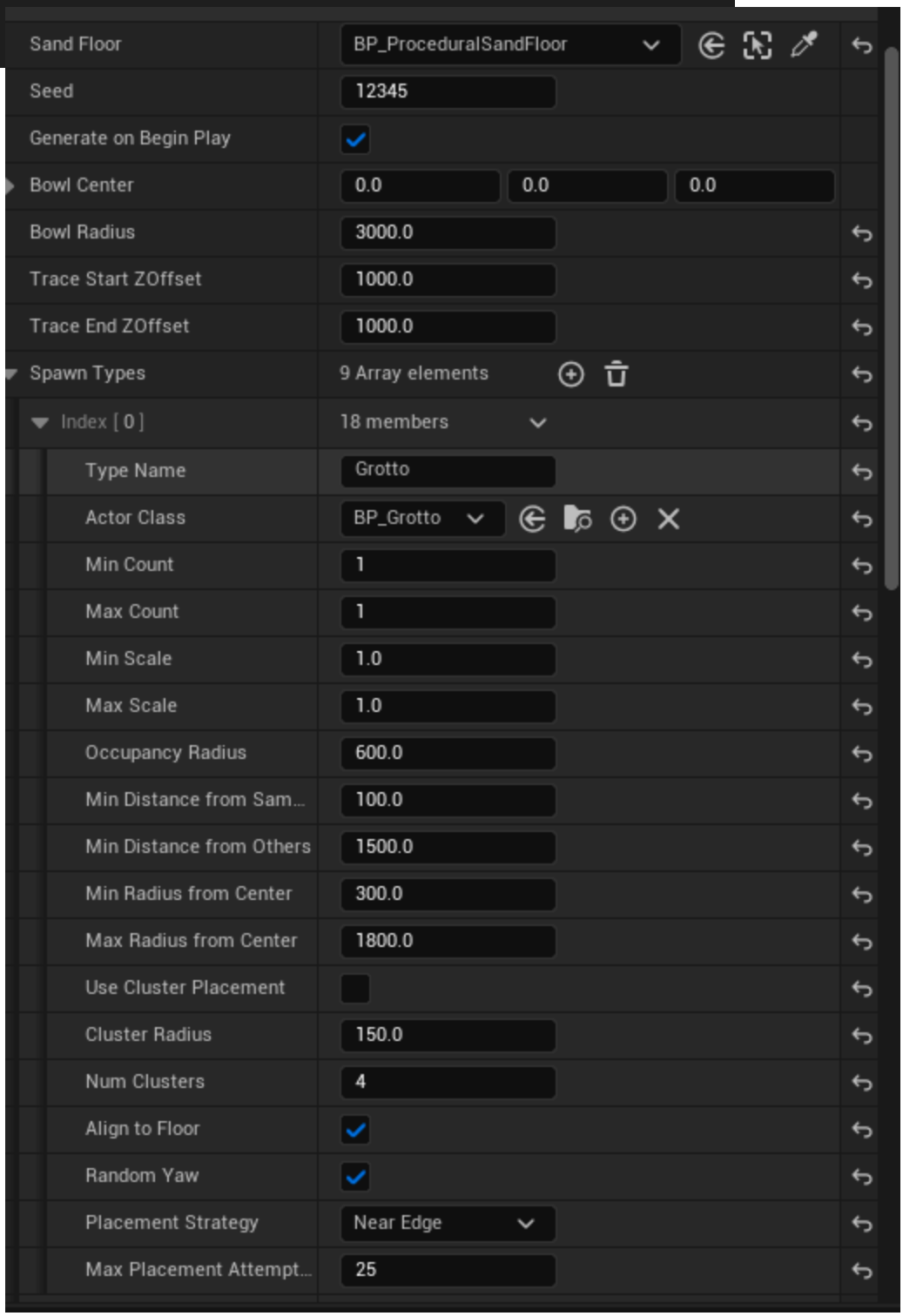
What patterns did you use?

I used the Strategy pattern and a data-driven Type Object approach.

The Strategy pattern is used for object placement behavior. Different objects are placed using different strategies, such as random scattering or clustered placement. For example, seaweed is spawned in small groups, while rocks and larger props are more evenly distributed. This allows the generator to support multiple placement behaviors without needing separate systems for each object type.

The Type Object pattern is partially implemented through structured spawn data. Each spawnable object has associated data such as spawn count, scale range, and spacing rules. While this is currently implemented using structs rather than Unreal Data Assets, it still separates object configuration from generation logic. This makes the system easier to extend and modify without rewriting core code.

```cpp
UENUM(BlueprintType)
enum class EPlacementStrategy : uint8
{
    RandomScatter UMETA(DisplayName = "Random Scatter"),
    Clustered     UMETA(DisplayName = "Clustered"),
    NearEdge      UMETA(DisplayName = "Near Edge")
};
```



```cpp
bool AFishBowlGenerator::TryPlaceObject(const FSpawnTypeData& SpawnType)
{
    UE_LOG(LogTemp, Warning, TEXT("Trying to place: %s"), *SpawnType.TypeName.ToString());

    if (!SpawnType.ActorClass)
    {
        return false;
    }

    for (int32 Attempt = 0; Attempt < SpawnType.MaxPlacementAttemptsPerInstance; ++Attempt)
    {
        FTransform CandidateTransform;
        FVector CandidateLocation;

        if (!GetCandidateTransform(SpawnType, CandidateTransform, CandidateLocation))
        {
            continue;
        }

        if (!IsCandidateValid(SpawnType, CandidateLocation))
        {
            continue;
        }

        AActor* NewActor = SpawnPlacedActor(SpawnType, CandidateTransform);
        if (!IsValid(NewActor))
        {
            return false;
        }

        FPlacedObjectInfo Info;
        Info.TypeName = SpawnType.TypeName;
        Info.Location = CandidateLocation;
        Info.OccupancyRadius = SpawnType.OccupancyRadius;
        Info.SpawnedActor = NewActor;

        PlacedObjects.Add(Info);
        SpawnedActors.Add(NewActor);

        return true;
    }

    return false;
}
```

What patterns did you choose not to use?

I chose not to use the full Type Object implementation using Data Assets, even though it would improve scalability. Since I was still iterating quickly, using structs inside the generator was faster and easier to debug. If the project were expanded, converting to Data Assets would be a clear next step.

I also chose not to use a more complex event-driven or observer-based system for generation. Instead, I kept the logic centralized in the generator. This avoided unnecessary complexity and made debugging easier. Given the scope of the project, adding more architectural layers would not have provided enough benefit.

```cpp
bool AFishBowlGenerator::GetCandidateTransform(const FSpawnTypeData& SpawnType, FTransform& OutTransform, FVector& OutLocation)
{
    float X;
    float Y;

    if (SpawnType.bUseClusterPlacement)
    {
        //RandomStream.Initialize(Seed + SpawnType.NumClusters * 17);

        TArray<FVector2D> ClusterCenters;
        for (int32 i = 0; i < SpawnType.NumClusters; i++)
        {
            const float ClusterAngle = RandomStream.FRandRange(0.0f, 2.0f * PI);
            const float ClusterDist = FMath::Sqrt(RandomStream.FRand()) * BowlRadius * 0.8f;

            const float CX = BowlCenter.X + FMath::Cos(ClusterAngle) * ClusterDist;
            const float CY = BowlCenter.Y + FMath::Sin(ClusterAngle) * ClusterDist;

            ClusterCenters.Add(FVector2D(CX, CY));
        }

        const int32 ClusterIndex = RandomStream.RandRange(0, ClusterCenters.Num() - 1);
        const FVector2D ClusterCenter = ClusterCenters[ClusterIndex];

        const float LocalAngle = RandomStream.FRandRange(0.0f, 2.0f * PI);
        const float LocalRadius = FMath::Sqrt(RandomStream.FRand()) * SpawnType.ClusterRadius;

        X = ClusterCenter.X + FMath::Cos(LocalAngle) * LocalRadius;
        Y = ClusterCenter.Y + FMath::Sin(LocalAngle) * LocalRadius;
    }
    else
    {
        const float Angle = RandomStream.FRandRange(0.0f, 2.0f * PI);
        const float Radius = FMath::Sqrt(RandomStream.FRand()) * BowlRadius;

        X = BowlCenter.X + FMath::Cos(Angle) * Radius;
        Y = BowlCenter.Y + FMath::Sin(Angle) * Radius;
    }

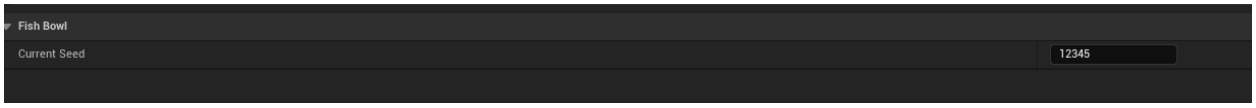
    FVector TerrainLocation;
    FVector TerrainNormal;
```

How did you feel about your implementation? What would you do differently?

I am satisfied with the implementation because it successfully generates visually distinct and playable environments from a seed. The system balances randomness and control, which helps the environment feel intentional rather than chaotic. The integration of gameplay, where the player collects chests and triggers regeneration, makes the procedural system meaningful rather than purely visual.

If I were to improve the project, I would first convert the spawn system into Data Assets to better support scalability and reuse. I would also implement multi-frame generation instead of generating everything in a single function call. This would allow for a proper loading screen or progress bar and improve perceived performance. Additionally, I would refine placement strategies for important objects so that key props feel more intentionally placed. Lastly, I would add more editable options, like in Minecraft, especially for bowl size, among other things

(making a fully fleshed-out type object that could be used in the menu similar to how I set it up in the editor).





Catch-All

One challenge I encountered was managing object overlap and ensuring that assets spawned on the terrain rather than on top of each other. I solved this by introducing spacing rules and priority-based spawning. Another issue was ensuring that seeds persisted correctly between level loads, which required storing the seed in the GameInstance.

A feature I am particularly happy with is the ability to input or randomize seeds from the menu, which gives players control over the procedural system and makes each run feel unique. As mentioned previously, I wanted to give more control, like how Minecraft allows, but something like that would take a lot of planning and implementation, which I do not currently have the time for

Time Spent

I spent approximately 30 hours on this project. The core generation systems took the bulk of my time, ranging from 8-10 hours, spent on refining and getting the generation to look as natural as possible. The UI and figuring out the game Instance to utilize the seed input from the menu took around 4-6 hours. Debugging and fully understanding the generation took a good 4 hours, while general polishing and environment work, like modeling the fishbowl and getting the underwater effect, as well as the background setting, took the remainder of the time.